- **2) Two-Address Instruction**

MOV	R1, A	$R1 \leftarrow M[A]$
ADD	R1, B	$R1 \leftarrow R1 + M[B]$
MOV	R2, C	$R2 \leftarrow M[C]$
ADD	R2, D	$R2 \leftarrow R2 + M[D]$
MUL	R1, R2	$R1 \leftarrow R1 * R2$
MOV	X, R1	$M[X] \leftarrow R1$

- The most common in commercial computers
- Each address fields specify either a processor register or a memory operand
- **3) One-Address Instruction**

LOAD	A	$AC \leftarrow M[A]$
ADD	B	$AC \leftarrow AC + M[B]$
STORE	T	$M[T] \leftarrow AC$
LOAD	C	$AC \leftarrow M[C]$
ADD	D	$AC \leftarrow AC + M[D]$
MUL	T	$AC \leftarrow AC * M[T]$
STORE	X	$M[X] \leftarrow AC$

- All operations are done between the AC register and memory operand

- 4) Zero-Address Instruction

PUSH	**A**	$TOS \leftarrow A$
PUSH	**B**	$TOS \leftarrow B$
ADD		$TOS \leftarrow (A + B)$
PUSH	**C**	$TOS \leftarrow C$
PUSH	**D**	$TOS \leftarrow D$
ADD		$TOS \leftarrow (C + D)$
MUL		$TOS \leftarrow (C + D) * (A + B)$
POP	**X**	$M[X] \leftarrow TOS$

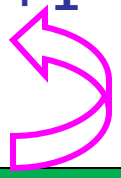
  - Stack-organized computer does not use an address field for the instructions **ADD**, and **MUL**
  - **PUSH**, and **POP** instructions need an address field to specify the operand
  - Zero-Address : absence of address (**ADD, MUL**)

– RISC Instruction
  - Only use **LOAD** and **STORE** instruction when communicating between memory and CPU
  - All other instructions are executed within the registers of the CPU without referring to memory
  - RISC architecture will be explained in *Sec. 8-8*

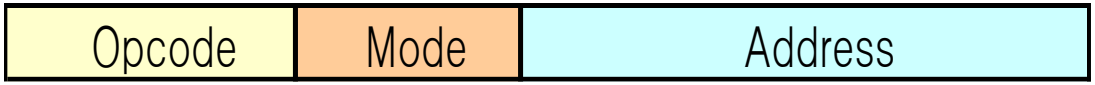
- Program to evaluate $X = (A + B) * (C + D)$

LOAD	R1, A	$R1 \leftarrow M[A]$
LOAD	R2, B	$R2 \leftarrow M[B]$
LOAD	R3, C	$R3 \leftarrow M[C]$
LOAD	R4, D	$R4 \leftarrow M[D]$
ADD	R1, R1, R2	$R1 \leftarrow R1 + R2$
ADD	R3, R3, R4	$R3 \leftarrow R3 + R4$
MUL	R1, R1, R3	$R1 \leftarrow R1 * R3$
STORE	X, R1	$M[X] \leftarrow R1$

- 8-5 Addressing Modes
  - Addressing Mode
    - 1) To give programming versatility to the user
      - pointers to memory, counters for loop control, indexing of data, ….
    - 2) To reduce the number of bits in the addressing field of the instruction
  - Instruction Cycle
    - 1) Fetch the instruction from memory and **PC + 1**
    - 2) Decode the instruction
    - 3) Execute the instruction



- Program Counter (***PC***)
  - PC keeps track of the instructions in the program stored in memory
  - PC holds the address of the instruction to be executed next
  - PC is incremented each time an instruction is fetched from memory
- Addressing Mode of the Instruction
  - 1) Distinct Binary Code
    - Instruction Format => Opcode | Addressing Mode Field
  - 2) Single Binary Code
    - Instruction Format => Opcode + Addressing Mode Field
- Instruction Format with mode field : ***Fig. 8-6***



- Implied Mode
  - Operands are specified implicitly in definition of the instruction
  - ***Examples***
    - **COM** : Complement Accumulator
      - » Operand in AC is implied in the definition of the instruction
    - **ADD**: Stack Operation
      - » Operands are implied to be on top of the stack

- Immediate Mode
  - Operand field contains the actual operand
  - Useful for initializing registers to a constant value
  - ***Example*** **: LD #NBR**
- Register Mode
  - Operands are in registers
  - Register is selected from a register field in the instruction
    - k-bit register field can specify any one of 2^k registers
  - ***Example*** : **LD R1** $AC \leftarrow R1$ Implied Mode
- Register Indirect Mode
  - Selected register contains the address of the operand rather than the operand itself
  - Address field of the instruction uses fewer bits to select a memory address
    - Register select bit
  - ***Example*** : **LD (R1)** $AC \leftarrow M[R1]$
- Autoincrement or Autodecrement Mode
  - Similar to the register indirect mode except that
    - the register is *incremented after* its value is used to access memory
    - the register is *decrement before* its value is used to access memory

- ***Example*** (*Autoincrement*) : **LD (R1)+** $AC \leftarrow M[R1], R1 \leftarrow R1+1$

- Direct Addressing Mode
  - Effective address is equal to the address field of the instruction (Operand)
  - Address field specifies the actual branch address in a branch-type instruction
  - ***Example*** : **LD ADR** $AC \leftarrow M[ADR]$

ADR = Address part of Instruction

- Indirect Addressing Mode
  - Address field of instruction gives the address where the effective address is stored in memory
  - ***Example*** : **LD @ADR** $AC \leftarrow M[M[ADR]]$
- Relative Addressing Mode
  - PC is added to the address part of the instruction to obtain the effective address
  - ***Example*** : **LD $ADR** $AC \leftarrow M[PC+ADR]$
- Indexed Addressing Mode
  - XR (*Index register*) is added to the address part of the instruction to obtain the effective address
  - ***Example*** : **LD ADR(XR)** $AC \leftarrow M[ADR+XR]$
- Base Register Addressing Mode
  - the content of a base register is added to the address part of the instruction to obtain the effective address

- Similar to the indexed addressing mode except that the register is now called a *base register* instead of an *index register*
  - index register (XR) : **LD ADR(XR)** $AC \leftarrow M[ADR + XR]$ ADR
    - » index register hold an index number that is relative to the address part of the instruction
  - base register (BR) : **LD ADR(BR)** $AC \leftarrow M[BR + ADR]$ BR
    - » base register hold a base address
    - » the address field of the instruction gives a displacement relative to this base address

– Numerical Example

Addressing Mode	Effective Address	Content of AC
Immediate Address Mode	201	500
Direct Address Mode	500	800
Indirect Address Mode	800	300
Register Mode		400
Register Indirect Mode	400	700
Relative Address Mode	702	325
Indexed Address Mode	600	900
Autoincrement Mode	400	700
Autodecrement Mode	399	450

R1 = 400

500 + 202 (**PC**)

500 + 100 (**XR**)

R1 = 400 + 1 (**after**)

R1 = 400 - 1 (**prior**)